

Kubernetes Interview Prep — Q&A

Prepared for Ketan Dhadve — DevOps Engineer Intern

Basic

1. What is Kubernetes, and what problem does it solve?

Kubernetes is an open-source container orchestration platform that automates deploying, scaling, healing, and networking containerized applications across a cluster of machines. It solves the problem of running containers reliably at scale — without it, you'd have to manually decide which server runs which container, restart crashed containers yourself, manually load-balance traffic, and manually scale up/down based on load.

2. What are the main components of the control plane (API Server, etcd, Scheduler, Controller Manager)?

The API Server is the front door to the cluster — every `kubectl` command and every internal component talks to it via REST/gRPC; it validates and processes requests. `etcd` is a distributed key-value store holding the entire cluster's state (every object's desired and current configuration) — it's the single source of truth. The Scheduler watches for newly created pods with no assigned node and decides which node they should run on, based on resource requests, affinity rules, taints, etc. The Controller Manager runs the various controllers (Deployment controller, ReplicaSet controller, Node controller, etc.) that continuously watch the actual cluster state and reconcile it toward the desired state stored in `etcd`.

3. What is a Node, and what runs on it (kubelet, kube-proxy, container runtime)?

A Node is a worker machine (physical or virtual) that runs application workloads. Three key components run on every node: the kubelet, an agent that talks to the API Server, ensures the containers described in assigned pods are actually running, and reports node/pod health back to the control plane; kube-proxy, which manages network rules on the node so traffic to a Service correctly reaches the right pod; and the container runtime (containerd, CRI-O) which actually pulls images and runs containers.

4. What is a Pod, and why is it the smallest deployable unit (not a container)?

A Pod is the smallest deployable unit in Kubernetes — a group of one or more containers that share the same network namespace (same IP, same localhost) and can share storage volumes. It's the unit, not the container, because some workloads genuinely need multiple tightly-coupled containers scheduled together and always co-located — e.g. a main application container plus a sidecar log-shipper — and Kubernetes needs an atomic unit to schedule, network, and manage as one.

5. What is a ReplicaSet?

A ReplicaSet ensures a specified number of identical pod replicas are running at all times — if a pod crashes or is deleted, the ReplicaSet controller notices the actual count no longer matches the desired count and creates a replacement. In practice, ReplicaSets are almost never created directly; they're managed automatically by a Deployment.

6. What is a Deployment, and how does it relate to a ReplicaSet?

A Deployment is a higher-level object that manages ReplicaSets on your behalf, adding rolling update and rollback capabilities. When you update a Deployment's pod template (e.g. a new image version), it creates a new ReplicaSet and gradually shifts replicas from the old ReplicaSet to the new one, keeping the old ReplicaSet around (scaled to zero) so you can roll back instantly if needed.

7. What is a Service, and why is it needed (pods are ephemeral)?

Pods are ephemeral — they get a new IP address every time they're recreated (after a crash, a rolling update, or rescheduling). A Service provides a stable virtual IP and DNS name in front of a dynamic set of pods (matched via a

label selector), so other parts of the application always have one consistent address to reach the current healthy pods, regardless of how many times the underlying pods have been replaced.

8. What are the Service types (ClusterIP, NodePort, LoadBalancer, ExternalName)?

ClusterIP (default) exposes the Service only inside the cluster, on an internal virtual IP — used for internal service-to-service communication. NodePort exposes the Service on a static port on every node's IP, making it reachable from outside the cluster at `<NodeIP>:<NodePort>`. LoadBalancer provisions an actual external load balancer from the cloud provider (e.g. an AWS ELB/NLB) pointing at the service, the most common way to expose a service to the public internet in a cloud-managed cluster. ExternalName maps a Service to an external DNS name (e.g. a managed database endpoint outside the cluster) via a CNAME, without proxying traffic through kube-proxy at all.

9. What is a Namespace?

A Namespace is a way to logically partition a single cluster into multiple virtual clusters — separating resources (pods, services, configmaps) by team, environment, or application, so names don't collide and RBAC/resource quotas can be scoped per namespace.

10. What is a ConfigMap?

A ConfigMap stores non-sensitive configuration data (key-value pairs, or whole config files) separately from application code/images, which can then be injected into pods as environment variables, command-line arguments, or mounted files — letting the same image be reused across environments with different configuration.

11. What is a Secret, and how is it different from a ConfigMap?

A Secret is structurally almost identical to a ConfigMap but intended for sensitive data (passwords, tokens, keys). The values are stored base64-encoded (not encrypted by default — base64 is just an encoding, not encryption) and Kubernetes can be configured with encryption-at-rest for Secrets in etcd. Kubernetes also avoids showing Secret values by default in `kubectl describe` output, unlike ConfigMaps which show plainly.

12. What is kubectl?

`kubectl` is the command-line client used to interact with a Kubernetes cluster's API Server — creating, inspecting, updating, and deleting resources, viewing logs, and executing commands inside pods.

13. How do you get all pods in a namespace?

`kubectl get pods -n <namespace>` — or `kubectl get pods --all-namespaces` (or `-A`) to see pods across every namespace.

14. How do you view logs of a pod?

`kubectl logs <pod-name>` — add `-f` to stream/follow logs live, and `-c <container-name>` when the pod has multiple containers, to specify which container's logs to view.

15. How do you describe a resource for troubleshooting (kubectl describe)?

`kubectl describe pod <pod-name>` shows detailed information beyond `kubectl get` — full status conditions, resource requests/limits, mounted volumes, and crucially the Events section at the bottom, which shows scheduling decisions, pull failures, probe failures, and other events that are often the fastest way to diagnose why a pod isn't working.

16. What is a label and a selector?

A label is an arbitrary key-value pair attached to an object (e.g. `app: flight-reservation`, `env: prod`) for identification and grouping purposes. A selector is a query that matches objects by their labels — used by Services to determine which pods to route to, by Deployments to know which pods they own, and by `kubectl` commands to filter output (`kubectl get pods -l app=flight-reservation`).

17. What is a manifest file (YAML) used for in Kubernetes?

A manifest is a YAML (or JSON) file that declaratively describes the desired state of a Kubernetes object — its kind, metadata, and spec. Applying it (`kubectl apply -f`) tells Kubernetes to create or update the actual cluster state to match what's described, forming the basis of Kubernetes' declarative, version-controllable configuration model.

18. What is `kubectl apply` vs `kubectl create`?

`kubectl create` creates a new resource from a manifest, but fails with an error if that resource already exists. `kubectl apply` is declarative — it creates the resource if it doesn't exist, or computes a diff and updates it in place if it does, making it safe to re-run repeatedly (idempotent) and the standard choice for GitOps/CI-driven workflows.

19. What is a `StatefulSet`, and how does it differ from a `Deployment`?

A `StatefulSet` manages pods that need a stable, unique identity and stable storage across restarts — each pod gets a predictable name (`pod-0`, `pod-1`, ...) and its own dedicated `PersistentVolumeClaim` that follows it even if it's rescheduled. A `Deployment`'s pods are interchangeable and disposable, with no guaranteed naming or storage stickiness. `StatefulSets` are used for things like databases or message queues where identity and data continuity matter; `Deployments` are used for stateless application replicas.

20. What is a `DaemonSet`?

A `DemonSet` ensures that a copy of a specific pod runs on every node in the cluster (or a selected subset via node selectors) — commonly used for node-level agents like log collectors, monitoring agents, or network plugins that need to run on every single node exactly once.

Intermediate / Scenario

21. What is a `Liveness Probe` vs a `Readiness Probe` vs a `Startup Probe`?

A `Liveness Probe` checks if the application is still alive/healthy — if it fails repeatedly, Kubernetes kills and restarts the container. A `Readiness Probe` checks if the application is ready to accept traffic — if it fails, the pod is removed from the `Service`'s endpoint list (traffic stops routing to it) without restarting the container, useful during temporary states like warming up a cache. A `Startup Probe` is used for slow-starting applications — it disables liveness/readiness checks until the startup probe itself succeeds, preventing Kubernetes from killing a container that's simply still starting up (which would otherwise look like a liveness failure).

22. What is a rolling update, and how do `maxSurge`/`maxUnavailable` control it?

A rolling update gradually replaces old pods with new ones (rather than deleting all old pods and creating all new ones at once), keeping the application available throughout the update. `maxSurge` controls how many extra pods above the desired replica count can be created temporarily during the rollout (e.g. `maxSurge: 1` with 3 replicas allows up to 4 pods briefly). `maxUnavailable` controls how many pods can be unavailable at once during the rollout (e.g. `maxUnavailable: 1` means at least 2 of 3 replicas stay up throughout). Together they tune the tradeoff between rollout speed and available capacity during deployment.

23. What is a `Horizontal Pod Autoscaler (HPA)`?

HPA automatically adjusts the number of pod replicas in a `Deployment`/`ReplicaSet`/`StatefulSet` based on observed metrics — most commonly CPU or memory utilization, but also custom or external metrics (e.g. queue length) via the metrics API. It continuously compares current metric values against a target (e.g. 'keep average CPU at 60%') and scales replicas up or down accordingly.

24. What is the `Cluster Autoscaler`, and how does it differ from HPA?

HPA scales the number of pod replicas within existing node capacity. The `Cluster Autoscaler` instead scales the number of nodes in the cluster itself — if pods can't be scheduled anywhere due to insufficient node resources, it provisions new nodes (e.g. adding EC2 instances to an EKS node group); if nodes are underutilized, it removes them. The two typically work together: HPA decides more pods are needed, and if there's no room to schedule them, `Cluster Autoscaler` adds capacity so they can actually run.

25. What is a Persistent Volume (PV) and Persistent Volume Claim (PVC)?

A PersistentVolume is a piece of actual storage in the cluster (an EBS volume, NFS share, etc.), provisioned either manually by an admin or dynamically. A PersistentVolumeClaim is a request for storage made by a user/pod, specifying size and access mode — Kubernetes binds the claim to a matching available PV. Pods then reference the PVC, not the PV directly, decoupling application configuration from the underlying storage implementation.

26. What is a StorageClass, and what is dynamic provisioning?

A StorageClass defines a 'class' of storage (e.g. gp3 SSD vs a slower HDD tier) along with the provisioner responsible for creating volumes on demand. Dynamic provisioning means that when a PVC requests storage referencing a StorageClass, Kubernetes automatically creates a matching PV (e.g. provisioning a new EBS volume) on the fly, rather than requiring an administrator to pre-create PVs manually ahead of time.

27. What is an Ingress, and why do you need an Ingress Controller?

An Ingress is an object that defines HTTP/HTTPS routing rules into the cluster — host-based and path-based routing to different Services, TLS termination, etc. — from a single entry point instead of provisioning a separate LoadBalancer Service per application. The Ingress resource itself is just a set of rules; it does nothing on its own. An Ingress Controller (e.g. NGINX Ingress Controller, AWS Load Balancer Controller) is the actual component that watches Ingress resources and implements the routing, typically by configuring a reverse proxy or provisioning a cloud load balancer.

28. What is a Pod Disruption Budget (PDB)?

A PDB limits how many pods of a given application can be voluntarily disrupted at once (e.g. during a node drain for a cluster upgrade, or manual eviction) — specifying either a minimum number/percentage that must stay available, or a maximum that can be unavailable at a time. This prevents a rolling node upgrade from accidentally taking down too many replicas of a critical service simultaneously.

29. What is RBAC in Kubernetes (Roles, ClusterRoles, RoleBindings)?

RBAC (Role-Based Access Control) controls who can perform which actions on which resources. A Role defines a set of permissions (e.g. 'can get/list pods') scoped to a single namespace. A ClusterRole defines permissions scoped cluster-wide (or reusable across namespaces). A RoleBinding grants a Role's permissions to a specific user, group, or ServiceAccount within one namespace; a ClusterRoleBinding grants a ClusterRole's permissions cluster-wide.

30. What is a ServiceAccount, and how is it used by pods?

A ServiceAccount is an identity that pods use to authenticate to the Kubernetes API Server (distinct from a human user account). Every pod runs under a ServiceAccount (default if none specified), and RBAC RoleBindings can grant that ServiceAccount specific permissions — e.g. a pod that needs to list other pods, or (in an EKS context, via IAM Roles for Service Accounts / IRSA) assume an AWS IAM role to access AWS services like S3 or Secrets Manager without static credentials.

31. What is a Custom Resource Definition (CRD)?

A CRD extends the Kubernetes API with a new, custom object type beyond the built-in ones (Pod, Deployment, Service, etc.) — e.g. defining a Database or Certificate resource. Once a CRD is registered, instances of that custom resource can be created and managed with kubectl just like any built-in object, and their actual behavior is typically implemented by a controller watching for them.

32. What is a Kubernetes Operator?

An Operator is a controller (often built around one or more CRDs) that encodes operational knowledge for managing a specific, often stateful application — automating tasks a human operator would otherwise do manually, like provisioning a database cluster, handling backups, performing version upgrades, or failover. Example: a PostgreSQL Operator that lets you declare a Postgres CustomResource and have the operator handle replica setup, failover, and backups automatically.

33. What is node affinity vs pod affinity/anti-affinity?

Node affinity controls which nodes a pod can be scheduled on, based on node labels (e.g. only schedule on nodes labeled `disktype: ssd`) — a more flexible, rule-based alternative to simple `nodeSelector`. Pod affinity/anti-affinity controls scheduling relative to other pods: affinity schedules a pod near pods matching certain labels (e.g. co-locate a cache with the app that uses it for lower latency), while anti-affinity schedules pods away from each other (e.g. spreading replicas of the same app across different nodes/availability zones for resilience).

34. What is a taint and toleration?

A taint is applied to a node to repel pods from being scheduled there unless they explicitly tolerate it (e.g. tainting a GPU node so only GPU workloads land there). A toleration is added to a pod's spec to allow it to be scheduled onto a node with a matching taint. Together they implement a 'nodes reject by default, pods opt in' model — the inverse of affinity, which is a 'pods request, nodes don't inherently reject' model.

35. What is Helm, and why use it instead of raw YAML manifests?

Helm is Kubernetes' package manager — it packages a set of related manifests (Deployment, Service, ConfigMap, etc.) into a reusable 'chart' with templated values, so the same chart can be installed multiple times with different configuration (e.g. per environment) by just changing a `values.yaml` file, instead of maintaining separate copies of near-identical YAML. It also tracks release history, enabling helm rollback to a previous release in one command, and manages upgrades as atomic, versioned operations.

36. How do you perform a rollback of a failed Deployment?

`kubectl rollout undo deployment/<name>` reverts to the previous `ReplicaSet` revision. `kubectl rollout history deployment/<name>` shows past revisions, and `kubectl rollout undo deployment/<name> --to-revision=<N>` rolls back to a specific earlier revision rather than just the immediately previous one.

37. What is the difference between `kubectl exec` and `kubectl logs`?

`kubectl logs` retrieves the `stdout/stderr` output already produced by a container — read-only, no interaction. `kubectl exec` runs a new command (often an interactive shell) inside an already-running container, letting you actively inspect the filesystem, run diagnostic commands, or interact with the process — the Kubernetes equivalent of `docker exec`.

38. How does Kubernetes DNS-based service discovery work?

Kubernetes runs a cluster-internal DNS service (CoreDNS) that automatically creates a DNS record for every Service, following the pattern `<service-name>.<namespace>.svc.cluster.local` (often just `<service-name>` is enough within the same namespace). Any pod can reach another Service simply by its DNS name, which CoreDNS resolves to the Service's stable ClusterIP, which kube-proxy then load-balances across the matching healthy pods.

39. What is a NetworkPolicy, and what problem does it solve?

By default, all pods in a Kubernetes cluster can freely communicate with all other pods, regardless of namespace — a `NetworkPolicy` restricts this by defining explicit rules for which pods can talk to which other pods (and on which ports), based on label selectors. It solves the problem of network segmentation/isolation in a multi-tenant or security-sensitive cluster, e.g. ensuring only the backend service's pods can reach the database's pods, not every pod in the cluster. `NetworkPolicies` require a CNI plugin that supports enforcement (not all do) to actually take effect.

40. How would you design zero-downtime deployments in Kubernetes?

Use a rolling update strategy with sensible `maxSurge`/`maxUnavailable` values so old pods aren't removed until new ones are confirmed healthy, configure a proper readiness probe so traffic is only routed to a new pod once it's actually ready to serve requests (not just started), set a `PodDisruptionBudget` so voluntary disruptions (node drains, cluster upgrades) can't take down too many replicas at once, and ensure the application handles `SIGTERM` gracefully with a reasonable `terminationGracePeriodSeconds` so in-flight requests can complete before a pod is killed during a rollout.

41. What is resource requests vs limits, and why are both important?

requests specify the minimum resources (CPU/memory) a container is guaranteed and is used by the Scheduler to decide which node has enough room to place the pod. limits specify the maximum resources a container is allowed to consume — exceeding a memory limit gets the container OOM-killed, and exceeding a CPU limit gets it throttled (not killed). Both matter: requests without limits can let one container starve others on a shared node; limits without accurate requests can cause the Scheduler to overcommit a node, leading to real resource contention even though scheduling looked fine on paper.

42. How would you troubleshoot a pod stuck in Pending state?

Run `kubectl describe pod <name>` and check the Events section — the most common causes are insufficient cluster resources (no node has enough CPU/memory to satisfy the pod's requests, in which case Cluster Autoscaler should eventually add a node if configured), an unsatisfiable node affinity/taint requirement (no node matches the required labels or tolerates the pod's requirements), or a PVC that hasn't bound yet, which blocks scheduling since the volume isn't ready.

43. What is the difference between a Job and a CronJob in Kubernetes?

A Job runs one or more pods to completion for a one-off task (e.g. a database migration script), retrying on failure up to a configured limit, and considered done once the specified number of successful completions is reached. A CronJob creates Jobs on a repeating schedule defined with standard cron syntax, used for recurring tasks like nightly backups or scheduled report generation.

44. How does Kubernetes handle a node failure?

The kubelet on a healthy node sends periodic heartbeats to the API Server; if a node stops reporting, the Node Controller marks it NotReady after a timeout. After a further grace period, pods on that node are marked for eviction, and if those pods are managed by a Deployment/ReplicaSet/StatefulSet, the relevant controller creates replacement pods on other healthy nodes to restore the desired replica count. Pods that were part of a StatefulSet retain their identity and (if using network-attached storage like EBS) can have their PVC reattached to the new node the replacement pod lands on.

45. What is the purpose of `kubectl port-forward`?

`kubectl port-forward pod/<name> 8080:80` creates a temporary tunnel from a local port on your machine directly to a port inside a specific pod, without needing to go through a Service or Ingress — useful for quickly debugging or accessing an application (or a database inside the cluster) directly from your laptop during development or troubleshooting.

46. How would you migrate a stateful application (e.g., a database) to run inside Kubernetes safely?

Use a StatefulSet (not a Deployment) so the database gets stable network identity and a dedicated PersistentVolumeClaim per replica that survives pod rescheduling. Provision storage via a StorageClass backed by durable, network-attached storage (e.g. EBS) rather than node-local disk, since node-local storage is lost if the pod is rescheduled to a different node. Plan the actual data migration carefully — typically via a backup/restore or logical replication from the existing database into the new Kubernetes-hosted instance — and validate with read replicas or a parallel-run period before cutting over production traffic, rather than migrating a live production database in a single irreversible step.

47. What is a sidecar container pattern, and give a real-world example?

A sidecar is a secondary container that runs alongside the main application container in the same pod, sharing its network and (optionally) storage, to add a supporting capability without modifying the main application itself. A common real-world example is a log-shipping sidecar (e.g. Fluent Bit) that tails the main container's log files from a shared volume and forwards them to a centralized logging system, or an Envoy proxy sidecar in a service mesh handling traffic encryption/routing transparently for the main application container.

48. How do you securely inject secrets into pods without exposing them in plain YAML?

Rather than hardcoding sensitive values directly into a Secret manifest committed to git, reference an external secrets manager: tools like External Secrets Operator or the AWS Secrets and Configuration Provider (ASCP) for the Secrets Store CSI Driver sync values from AWS Secrets Manager/SSM Parameter Store into a native Kubernetes Secret or directly mount them as a volume at runtime, so the actual plaintext value never has to live in a YAML file in version control. Where possible, pods should also use IRSA (IAM Roles for Service Accounts) to access AWS services directly, avoiding the need to store AWS credentials as Kubernetes Secrets at all.

49. What is the difference between Kubernetes-managed load balancing (Service) and an Ingress Controller's load balancing?

A Service (via kube-proxy) load-balances traffic at Layer 4 (TCP/UDP) across the pods matching its selector, with no awareness of HTTP semantics like hostnames or URL paths. An Ingress Controller operates at Layer 7 (HTTP/HTTPS), understanding host and path-based routing rules, and can do things a plain Service can't — TLS termination, URL rewriting, routing different paths of the same domain to different backend Services — before ultimately forwarding traffic to the appropriate Service(s), which then handle the final Layer 4 distribution to individual pods.

50. How would you design a multi-tenant Kubernetes cluster with resource isolation?

Give each tenant its own Namespace, apply ResourceQuotas per namespace to cap total CPU/memory/object counts a tenant can consume, set sensible default requests/limits via LimitRange so a single misbehaving pod can't starve others, use RBAC RoleBindings scoped per namespace so tenants can only manage their own resources, and apply NetworkPolicies to prevent cross-tenant network access by default. For stronger isolation than namespace-level soft multi-tenancy provides, dedicated node pools per tenant (using taints/tolerations and node affinity) or separate clusters entirely may be warranted for tenants with strict compliance or noisy-neighbor concerns.

Troubleshooting

51. A pod is stuck in CrashLoopBackOff. Walk through your debugging steps.

Start with `kubectl logs <pod> --previous` to see the logs from the last crashed instance (the current instance may not have produced enough output yet). Check `kubectl describe pod <pod>` for the Events section and the last exit code/reason. Common causes: the application crashes on startup due to a missing environment variable or misconfigured connection string (this matches your own RDS endpoint mismatch issue causing CrashLoopBackOff), a failing liveness probe repeatedly killing an otherwise-working container, or insufficient memory causing an immediate OOM kill on startup.

52. A pod shows ImagePullBackOff. What are the possible causes?

The image tag doesn't exist or is misspelled in the manifest, the image is in a private registry and the pod's `imagePullSecrets` aren't configured (or the ServiceAccount lacks pull credentials), the node can't reach the registry due to network/DNS issues, or (in a multi-arch scenario) the image doesn't have a variant matching the node's CPU architecture.

53. Pods are Running but the application is unreachable via the Service. What would you check?

Confirm the Service's selector labels actually match the pod's labels (a common typo-driven mismatch) via `kubectl get endpoints <service>` — if it shows no endpoints, the selector isn't matching any pods. Check that the Service's `targetPort` matches the actual port the container listens on (not just the container's declared `containerPort`). Also verify the application inside the container is binding to `0.0.0.0` and not only `127.0.0.1`, since binding to `localhost` only would make it unreachable even from within the same pod's Service.

54. A Deployment rollout is stuck and never completes. How do you investigate?

`kubectl rollout status deployment/<name>` shows how many replicas are updated/available. `kubectl describe deployment <name>` and `kubectl get replicaset`s show whether new pods are even being created. If new pods exist

but aren't becoming Ready, check their readiness probe configuration and logs — a rollout waits indefinitely for new pods to pass readiness before proceeding, so a probe pointed at the wrong path/port, or an application that never actually becomes healthy, will stall the rollout forever without an explicit failure.

55. A pod was OOMKilled. How do you diagnose and fix it?

`kubectl describe pod <pod>` shows 'OOMKilled' as the last termination reason if this occurred. Check `kubectl top pod` (if metrics-server is installed) for actual memory usage trends leading up to the kill, and compare against the pod's configured memory limit. Fix by either raising the memory limit if usage is legitimate, or investigating the application itself for a memory leak (for a JVM-based app like your Spring Boot services, also check that `-Xmx` heap settings leave headroom under the container's memory limit, since JVM overhead beyond the heap needs to fit too).

56. `kubectl get nodes` shows a node as NotReady. What steps do you take?

`kubectl describe node <name>` shows the specific condition causing NotReady (e.g. kubelet not reporting, disk pressure, memory pressure, network plugin issues). If it's a cloud-managed node (e.g. EKS), check the underlying EC2 instance's health/status checks directly in the AWS console — the instance itself may have failed, be unreachable due to a security group/networking change, or have run out of disk space. If the kubelet process itself has stopped or crashed on an otherwise-healthy instance, SSH access (if available) to check kubelet's own logs/status is the next step.

57. A ConfigMap was updated, but the running pod isn't reflecting the new values. Why, and how do you fix it?

If the ConfigMap is consumed as environment variables, updating the ConfigMap does not automatically update already-running containers — environment variables are only set once at container start. If it's mounted as a volume, updates do eventually propagate to the mounted files (with some delay via kubelet sync), but the application itself may not re-read the file without a restart. The reliable fix in both cases is to trigger a rollout of the pods that consume it (e.g. `kubectl rollout restart deployment/<name>`), or use a tool like Reloader that watches ConfigMaps/Secrets and automatically triggers a rolling restart of dependent Deployments when they change.

58. Two pods can't communicate despite being in the same namespace. What would you check (NetworkPolicy, Service config)?

Check for a NetworkPolicy scoped to either pod's namespace or matching labels that might be restricting ingress/egress traffic — a default-deny policy without an explicit allow rule for the other pod would silently block traffic. If going through a Service rather than pod IP directly, verify the Service's selector and port mapping as in question 53. Also confirm both pods are actually in the same namespace and not just visually similar names in different namespaces, since cross-namespace communication requires the fully qualified DNS name.

59. HPA isn't scaling despite high CPU usage. What could be misconfigured?

Check `kubectl describe hpa <name>` for its current status and any error conditions. A very common cause is that the pods don't have resource requests defined at all — HPA's CPU percentage target is calculated relative to the requested CPU, so without a request, the utilization percentage can't be computed and scaling silently doesn't happen. Also confirm metrics-server (or the relevant custom metrics adapter) is actually installed and running in the cluster, since HPA depends entirely on it for utilization data.

60. A PVC is stuck in Pending and never binds to a PV. What's the likely cause?

With dynamic provisioning, check that the StorageClass referenced by the PVC actually exists and its provisioner is functioning (e.g. check the provisioner's controller pod logs for errors, such as an AWS IAM permissions issue preventing EBS volume creation). Also check whether the requested access mode or size can actually be satisfied — e.g. requesting ReadWriteMany on a storage backend (like standard EBS) that only supports ReadWriteOnce will never bind. With static provisioning, confirm a matching PV with the right size/access mode/StorageClass actually exists and isn't already bound to a different claim.

61. An Ingress resource is created, but traffic isn't being routed correctly. What would you check?

Confirm an Ingress Controller is actually installed and running in the cluster — an Ingress resource with no controller watching it does nothing at all. Check that the `ingressClassName` in the Ingress spec matches the installed controller. Verify the host/path rules match what's actually being requested, and that the backend Service name/port referenced in the Ingress is correct and has healthy endpoints. Finally check the Ingress Controller's own logs/events, since routing errors (like a misconfigured rewrite rule) usually surface there rather than on the Ingress object itself.

62. A node drain during a cluster upgrade caused unexpected downtime. What Kubernetes features would prevent this in future (PDB, replicas, readiness probes)?

A `PodDisruptionBudget` ensures the drain process can't evict too many replicas of the same application at once, forcing the drain to proceed more slowly and only when enough replicas remain available elsewhere. Running enough replicas (at least 2-3) spread across multiple nodes/availability zones via pod anti-affinity means losing pods on one drained node doesn't take down the whole application. Proper readiness probes ensure traffic only shifts to newly scheduled replacement pods once they're actually ready, rather than briefly routing to pods that are still starting up during the disruption.

63. `kubectl` commands are timing out against the cluster. What could be wrong (API server health, network, kubeconfig, RBAC)?

Check whether the API Server itself is healthy and reachable — for a managed control plane like EKS, check the AWS console/CloudWatch for control plane health issues. Verify network connectivity/DNS resolution to the cluster endpoint from wherever `kubectl` is running (VPN, security group rules, or a private API endpoint requiring access from within the VPC). Check that the kubeconfig file points at the correct, current cluster endpoint and hasn't gone stale after a cluster recreation. Finally, an RBAC/IAM authentication issue (e.g. the credentials being used don't map to any Kubernetes RBAC identity, similar to the IAM credential precedence issue you've debugged before) typically produces an immediate 'Unauthorized' error rather than a timeout, so a genuine timeout more often points to a network or control-plane availability issue than to permissions.

64. A `StatefulSet` pod lost its persistent data after a restart. What misconfiguration could cause this?

The most likely cause is that the pod's volume was backed by node-local ephemeral storage (`emptyDir` or a `hostPath`) instead of a proper `PersistentVolumeClaim` backed by durable, network-attached storage — ephemeral storage is tied to the specific node and pod instance and is lost when the pod is rescheduled. Another possibility is the `StatefulSet`'s `volumeClaimTemplates` being misconfigured or removed, or the reclaim policy on the underlying PV being set to `Delete` instead of `Retain`, causing the actual storage volume to be deleted when the PVC was deleted or recreated during troubleshooting.

65. Cluster-wide DNS resolution (CoreDNS) is failing intermittently. How do you troubleshoot?

Check the CoreDNS pods themselves (`kubectl get pods -n kube-system -l k8s-app=kube-dns`) for restarts, crashes, or resource throttling — CoreDNS pods being CPU-throttled under load is a common cause of intermittent (not total) DNS failures. Check CoreDNS's own logs for errors reaching upstream DNS servers. Also check whether CoreDNS's replica count and resource requests/limits are appropriately sized for cluster DNS query volume — a cluster that has grown significantly in pod count without scaling CoreDNS accordingly can start seeing timeouts under load, which is fixed by increasing CoreDNS replicas or resource allocation.